

Akıllı Kargo ve Sipariş Yönetim Sistemi Proje Raporu

Giriş

Bu proje, e-ticaret platformlarının temel bileşenlerini yönetmek üzere tasarlanmış nesne yönelimli bir yazılım sistemi geliştirmeyi amaçlamaktadır. Geliştirilen sistem, sipariş, ürün, müşteri ve kargo süreçlerini kapsayarak, kullanıcıların ürün sipariş edebildiği, sistemin en uygun kargo seçeneğini belirlediği ve teslimat sürecini takip edebildiği modüler, genişletilebilir ve bakımı kolay bir mimari sunmaktadır.

Yazılım, ürün yönetimi (yeni ürün ekleme, listeleme, stok takibi ve kategoriye göre filtreleme), müşteri işlemleri (kayıt, profil oluşturma, sipariş geçmişi görüntüleme), sipariş sistemi (sipariş oluşturma, durumu güncelleme), kargo yöntemleri (farklı stratejiler ve otomatik seçim) ve bildirim mekanizması gibi temel fonksiyonel istekleri karşılamaktadır. Ayrıca, siparişle birlikte stok takibi yapılmakta ve stokta olmayan ürünlerin sipariş edilmesi engellenmektedir.

Ana Sınıflar ve Amaçları

Projede, sistemin temel varlıklarını ve işlevlerini temsil eden nesne yönelimli sınıflar kullanılmıştır. Bu sınıfların başlıcaları ve amaçları aşağıda açıklanmıştır:

- OrderStatus (Enum):

Amaç: Siparişin olası durumlarını (Beklemede, İşleniyor, Gönderildi, Teslim Edildi, İptal Edildi) standartlaştırmak ve kod içinde tutarlı bir şekilde kullanılmasını sağlamak. Bir siparişin anlık durumunu belirlemek için kullanılır.

- Product:

Amaç: Sistemdeki her bir ürünü temsil etmek. Ürünün ID'si, adı, fiyatı, mevcut stok miktarı ve kategorisi gibi temel bilgilerini kapsüller. Ürün bilgilerini görüntüleme ve stok miktarını güncelleme gibi işlemlerden sorumludur.

- Customer:

Amaç: Sisteme kayıtlı her bir müşteriye temsil etmek. Müşterinin ID'si, adı, e-posta adresi, fiziksel adresi gibi kişisel bilgilerini ve sipariş geçmişini yönetir. Müşteri profili görüntüleme ve yeni siparişleri geçmişine ekleme işlevlerini sağlar.

- Order:

Amaç: Müşteriler tarafından verilen her bir siparişi temsil etmek. Siparişin benzersiz ID'si, ilgili müşteri ID'si, sipariş edilen ürünler ve miktarları, sipariş tarihi, güncel durumu, kullanılan kargo yöntemi ve toplam tutarı gibi bilgileri

içerir. Siparişin durumunu güncelleme, detaylarını görüntüleme ve iptal etme gibi işlemlerden sorumludur.

- **ShippingMethod (Soyut Sınıf):**

Amaç: Farklı kargo yöntemleri için ortak bir arayüz tanımlamak. Kargo maliyetini hesaplama işlevini soyutlar. Bu, farklı kargo stratejilerinin (FastShipping, EconomicShipping, DroneShipping) kendi maliyet hesaplama algoritmalarını implemente etmesini sağlar.

- **FastShipping, EconomicShipping, DroneShipping (Somut Kargo Sınıfları):**

Amaç: Belirli kargo yöntemlerinin detaylarını ve maliyet hesaplama mantığını somutlaştırmak. ShippingMethod sınıfından türeyerek kendi spesifik kargo ücreti hesaplamalarını yaparlar.

- **InventoryManager:**

Amaç: Sistemdeki tüm ürün envanterini merkezi olarak yönetmek. Ürün bilgilerini yükleme, kaydetme, yeni ürün ekleme, stok kontrolü yapma ve stok miktarlarını güncelleme gibi kritik envanter operasyonlarından sorumludur. Singleton deseni ile tek bir örnek olarak çalışır ve stok verilerinin tutarlılığını sağlar.

- **NotificationService:**

Amaç: Sipariş durumu değişiklikleri gibi olaylarda ilgili müşterilere bildirim gönderme mekanizmasını sağlamak. Abone olan müşterilere (gözlemcilere) otomatik olarak mesaj iletmekten sorumludur.

- **OrderFactory:**

Amaç: Yeni Order nesnelerinin oluşturulması sürecini soyutlamak ve merkezi bir noktadan yönetmek. Sipariş oluşturulurken gerekli stok kontrolü, kargo maliyeti hesaplama gibi adımları kapsayarak siparişin doğru ve tutarlı bir şekilde başlatılmasını sağlar.

Fonksiyonel İsterler ve Gerçekleştirme

Projede belirtilen fonksiyonel isterler aşağıdaki gibi gerçekleştirilmiştir:

- **Ürün Yönetimi:**

Yeni ürün ekleme, listeleme ve stok bilgisi tutma: Product sınıfı ürün bilgilerini ve stok miktarını tutar. InventoryManager sınıfı ürünlerin yüklenmesi, kaydedilmesi, listelenmesi, stok kontrolü ve güncellenmesi işlemlerini merkezi olarak yönetir.

Ürün kategorilerine göre filtreleme: InventoryManager sınıfının list_products metodu kategoriye göre filtreleme yapabilmektedir.

- Müşteri İşlemleri:

Müşteri kaydı ve profil oluşturma: Customer sınıfı kullanıcı bilgilerini tutar. register_customer fonksiyonu yeni müşteri kaydı yapılmasını sağlar.

Sipariş geçmişini görüntüleme: Customer sınıfı order_history niteliği ile müşteriye ait sipariş ID'lerini saklar. view_order_history fonksiyonu müşterinin sipariş geçmişini listeler.

- Sipariş Sistemi:

Müşteri ürün seçip sipariş oluşturabilmeli: Order sınıfı sipariş bilgilerini, ürün listesini ve durumunu tutar. place_order fonksiyonu, kullanıcının ürün seçerek sepet oluşturmalarını ve sipariş vermesini sağlar.

Sipariş durumu güncellenebilmeli: Order sınıfının update_status metodu ve update_order_status_cli fonksiyonu ile sipariş durumu güncellenebilmektedir.

- Kargo Yöntemleri:

Farklı kargo stratejileri (hızlı, ekonomik, drone): ShippingMethod soyut sınıfı ve FastShipping, EconomicShipping, DroneShipping alt sınıfları farklı kargo yöntemlerini temsil eder.

En uygun kargo yöntemi otomatik seçilmeli: place_order fonksiyonu, farklı kargo yöntemlerinin tahmini maliyetlerini gösterir ve kullanıcı seçim yapabilir. (Proje tanımında otomatik seçim yerine kullanıcı seçimi baz alınmıştır, ancak Strategy deseni ile farklı maliyetlendirme algoritmaları sunulmuştur.)

- Bildirim Mekanizması: Sipariş durum değişikliklerinde müşteriye bildirim gönderilmeli: NotificationService sınıfı bu işlevi yerine getirir.

- Stok Takibi: Siparişle birlikte stok güncellenmeli; stokta olmayan ürünler sipariş edilememeli: InventoryManager sınıfı check_stock ve update_stock metotları aracılığıyla stok takibini ve güncellemesini yönetir. OrderFactory sipariş oluşturulurken stok kontrolünü yapar ve stoğu günceller.

Yazılımda Kullanılan Tasarım Desenleri

Projede dört ana tasarım deseni kullanılmıştır ve bunların kullanım amaçları aşağıda detaylandırılmıştır.

1. Strategy Deseni (Kargo Yöntemleri):

Kullanım Amacı: Farklı kargo yöntemlerinin (Hızlı, Ekonomik, Drone vb.) dinamik olarak seçilebilmesini sağlamak ve her bir yöntemin maliyet hesaplama algoritmasını birbirinden bağımsız hale getirmektir.

Nasıl Kullanıldı: ShippingMethod adında soyut bir temel sınıf oluşturulmuştur. Bu sınıf, calculate_cost adında soyut bir metoda sahiptir. FastShipping, EconomicShipping ve DroneShipping adında somut strateji sınıfları ShippingMethod sınıfından türetilmiştir. Her somut sınıf, kendi özgü maliyet hesaplama mantığını calculate_cost metodunda uygular. Sipariş oluşturulurken (place_order fonksiyonunda), kullanıcı seçimine göre uygun ShippingMethod nesnesi seçilir ve bu nesne aracılığıyla kargo maliyeti hesaplanır. Bu sayede yeni bir kargo yöntemi eklemek veya mevcut bir yöntemin maliyet hesaplama mantığını değiştirmek, diğer kodları etkilemeden kolayca yapılabilir.

2. Observer Deseni (Bildirim Mekanizması):

Kullanım Amacı: Sipariş durumu değiştiğinde (konu), müşterilere (gözlemciler) otomatik olarak bildirim gönderilmesini sağlamaktır. Konu ve gözlemciler arasındaki bağımlılığı azaltır.

Nasıl Kullanıldı: NotificationService sınıfı hem bir konu (subject) hem de bir gözlemci (observer) gibi davranır. Müşteriler (Customer nesneleri), NotificationService'e attach metodu ile abone olabilirler. Sipariş durumu Order sınıfında güncellendiğinde veya update_order_status_cli fonksiyonu aracılığıyla bir siparişin durumu değiştiğinde, NotificationService'in notify metodu çağrılır ve ilgili müşteriye bildirim gönderilir. Bu, sipariş durumundaki değişikliklerin müşteri sistemine gevşek bağlı bir şekilde iletilmesini sağlar.

3. Singleton Deseni (Envanter Yöneticisi):

Kullanım Amacı: Tüm ürün envanterini yöneten InventoryManager sınıfından sadece bir tane örnek (instance) oluşturulmasını sağlamaktır. Böylece, stok verilerinin tutarlılığı merkezi bir noktadan garanti altına alınır.

Nasıl Kullanıldı: InventoryManager sınıfı, __new__ metodunu override ederek Singleton deseni uygulanmıştır. Bu metod, sınıfın örneği daha önce oluşturulmamışsa yeni bir örnek oluşturur, aksi takdirde mevcut örneği döndürür. Bu, uygulamanın herhangi bir yerinden InventoryManager() çağrıldığında her zaman aynı _instance üzerinde çalışmasını sağlar. Ürün verileri (JSON dosyalarından yüklenen ve kaydedilen) bu tek örnek tarafından yönetilir, böylece stok durumu her zaman doğru ve senkronize kalır.

4. Factory Method Deseni (Sipariş Fabrikası):

Kullanım Amacı: Yeni Order nesneleri oluşturma sürecini soyutlamak ve esnek hale getirmektir. Sipariş oluşturma mantığını (OrderFactory sınıfı) siparişi talep eden istemci koddan ayırır.

Nasıl Kullanıldı: OrderFactory adında statik metot içeren bir sınıf oluşturulmuştur. create_order statik metodu, siparişin oluşturulması için gerekli tüm adımları (stok kontrolü, kargo maliyeti hesaplama, ID oluşturma, tarih kaydetme vb.) kapsar. İstemci (CLI'daki place_order fonksiyonu), doğrudan Order nesnesini örneklemek yerine OrderFactory.create_order() metodunu çağırır. Bu sayede, gelecekte farklı sipariş türleri veya sipariş oluşturma süreçlerinde değişiklikler olduğunda, bu değişiklikler yalnızca fabrika sınıfında yapılır ve istemci kod etkilenmez.

Sonuç

Geliştirilen Akıllı Kargo ve Sipariş Yönetim Sistemi, nesne yönelimli programlama prensiplerini ve tasarım desenlerini etkin bir şekilde kullanarak e-ticaret platformunun temel sipariş akışını simüle eden modüler ve genişletilebilir bir yapı sunmaktadır.

Artıları:

- Modülerlik ve Genişletilebilirlik: Tasarım desenleri sayesinde yeni kargo yöntemleri, bildirim mekanizmaları veya ürün türleri eklemek sisteme kolayca entegre edilebilir. Özellikle Strategy ve Factory Method desenleri bu konuda esneklik sağlamıştır.
- Kod Organizasyonu ve Okunabilirlik: Sınıfların ve fonksiyonların belirli sorumluluklara sahip olması, kodun anlaşılabilirliğini ve bakımını kolaylaştırmaktadır.
- Veri Tutarlılığı: Singleton deseni ile InventoryManager'ın tek bir örnek olarak çalışması, stok verilerinin her zaman tutarlı olmasını sağlar.
- Bağımlılıkların Azaltılması: Observer deseni, sipariş durumu değişiklikleriyle müşteri bildirimi arasındaki doğrudan bağımlılığı azaltarak daha esnek bir iletişim yapısı sunar.
- OOP Prensiplerinin Uygulanması: Kapsülleme (Product, Customer, Order sınıflarının nitelikleri), veri soyutlama (ShippingMethod soyut sınıfı), polimorfizm (farklı ShippingMethod alt sınıflarının calculate_cost metotları) ve miras (ShippingMethod alt sınıfları) gibi OOP teknikleri düzgün bir şekilde aktarılmıştır.

Eksileri ve Geliştirme Alanları:

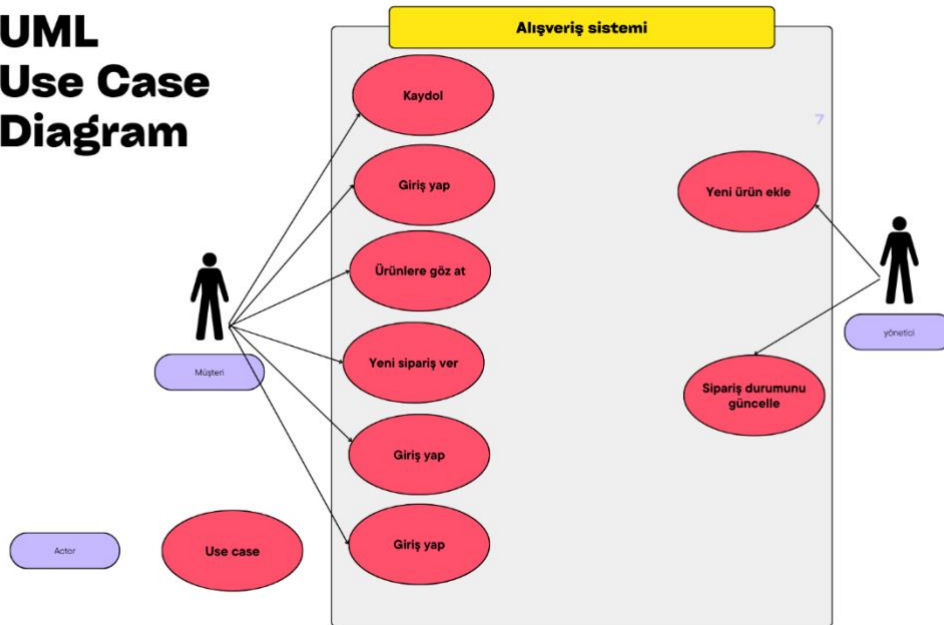
- Basit CLI Arayüzü: Mevcut sistem, basit bir terminal arayüzü ile çalışmaktadır. Gerçek bir e-ticaret platformu için daha zengin bir Grafik Kullanıcı Arayüzü (GUI) veya web arayüzü gerekebilir.

- **Hata Yönetimi ve Sağlamlık:** Kullanıcı girişlerindeki daha karmaşık senaryolar ve olası dosya okuma/yazma hataları için daha kapsamlı bir hata yönetimi eklenebilir. Örneğin, dosya bozulmaları veya ağ hataları gibi durumlar için daha sağlam çözümler düşünülebilir.
- **Güvenlik:** Müşteri giriş/çıkış sistemi temel bir kimlik doğrulama veya yetkilendirme mekanizması içermemektedir. Gerçek bir sistemde, kullanıcı yetkilendirme (müşteri/yönetici rolleri gibi) ve veri güvenliği önlemleri (parola şifreleme vb.) gereklidir.
- **Veri Saklama Yöntemi:** Veriler şu an için basit JSON dosyalarında saklanmaktadır. Daha büyük ölçekli ve performans odaklı uygulamalar için bir veritabanı (SQL veya NoSQL) çözümü daha uygun olacaktır.
- **İleri Kargo Hesaplamaları:** Kargo maliyetleri için sadece mesafe ve toplam ağırlık/hacim gibi basit faktörler kullanılmıştır. Gerçek dünya senaryolarında bölgeler, taşıyıcı anlaşmaları, aciliyet gibi daha fazla faktör maliyeti etkileyebilir.

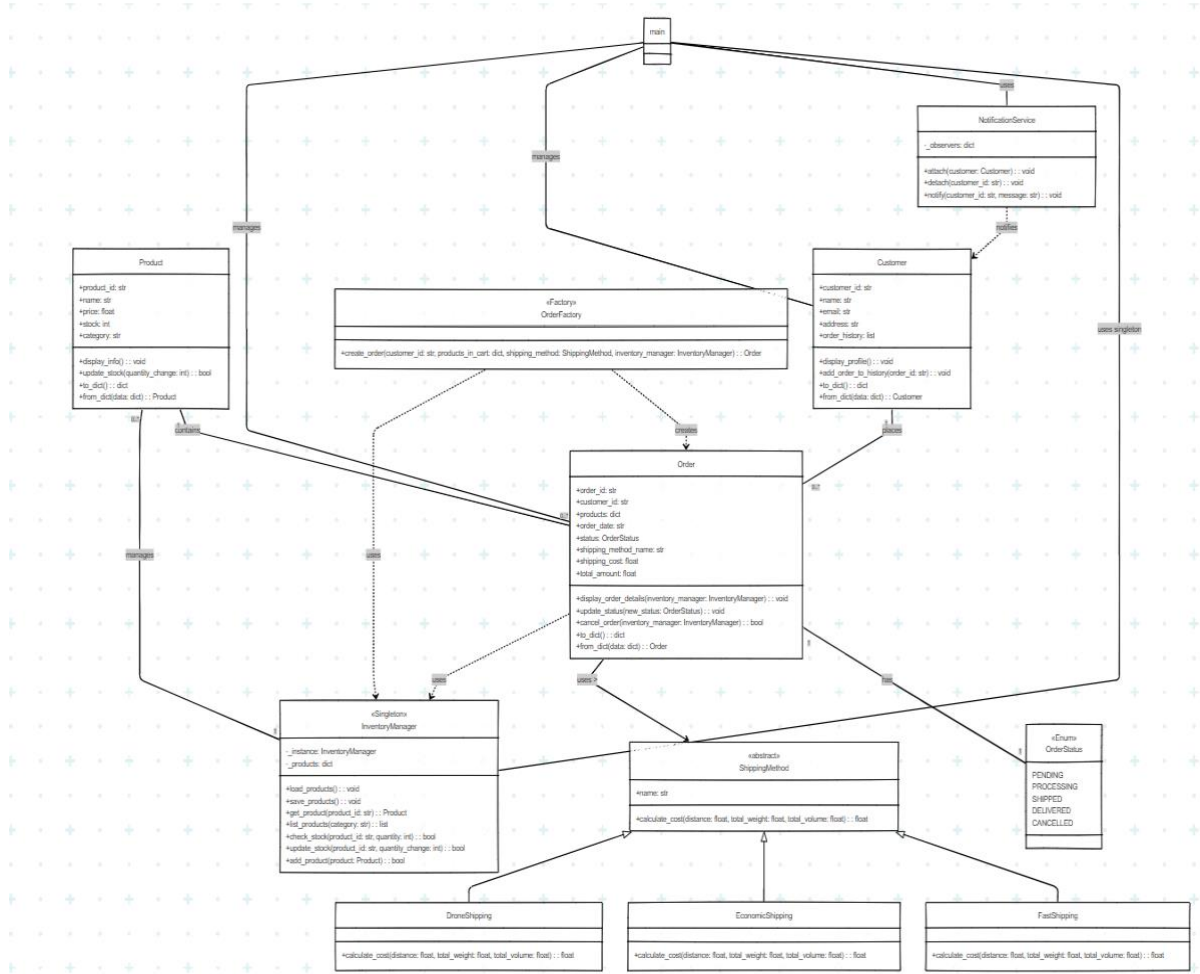
UML Sınıf Diyagramı ve Use Case Diyagramı

- **Use Case Diyagramı:**

UML Use Case Diagram



- UML Sınıf Diyagramı:



Grup Üyeleri:

Kadir Barış İNCEOĞLU (Sözcü) 21052035

Eren HACER 21052037

Gülendem ARICI 21052018

Muhamed Kutbi HAN 21052038